

LLM_Feedback

PROMPT:

" Please read over this summary i wrote up: Chicago Neighborhood Desirability Analysis

This analysis examines what drives neighborhood desirability in Chicago by using 17 observable neighborhood variables to predict home values. Neighborhood desirability is determined by the Zillow Home Value Index because home values are indicative of how safe, desirable, and economically viable an area is to its residents and investors. The objective was to determine what factors are most important to residential property values in Chicago's 53 community areas and to determine which machine learning model provided the most useful results for urban planning.

The data was obtained from a variety of public sources, including Zillow home values from 2000 to 2024, crime statistics through 2025, food inspections, parks, and Airbnb listings from 2023 to 2025. The predictor variables included crime statistics (total crimes, violent crimes, violent crime rate), dining variables (restaurant count, high-risk restaurants, food safety ratios), recreational facilities (park size), service demand (311 calls and density), Airbnb data (listing count, average price, occupancy, revenue, ratings, market penetration, revenue efficiency), and past home value growth. There were 53 neighborhoods in Chicago, with 39 used for training (75%) and 14 for testing (25%). Two machine learning models were used: a Random Forest Regressor and an XGBoost Gradient Boosting Model.

It is clear that XG Boost performed significantly better than Random Forest. The XG Boost algorithm had a high R-squared value of 0.774 for training data and 0.709 for test data. The algorithm also had a mean absolute error of \$65,574 and a Root Mean Squared Error of \$83,250. The difference between the test and training error is only 6.5%. On the other hand, Random Forest had a test R-squared value of 0.315, implying that it only explained 0.315 of the variance for home values in the Chicago neighborhood. The algorithm also had a test MAE of \$109,519. The difference between test and training error is 65%, which is a significant difference. The feature importance plot shows that the most important factor affecting home values is the average Airbnb rating, with a value of 0.23. The second most important factor is home value growth over time, with a value of 0.20. The third most important factor is high-risk business density, with a value of 0.16. The last most important factor is violent crime rate, with a value of 0.1475. The rest of the factors are less significant and have values less than 0.06.

The comparison between the two models shows the trade-off between interpretability and model performance. The Random Forest model is simple and interpretable but only

explains 0.315 test R-squared, meaning most differences between neighborhoods remain unexplained. On the other hand, the XGBoost model explains non-linear thresholds, interactions, and diminishing returns, such as the amplified effect of violent crime rates as they increase, the disproportionate effect of high-risk commercial areas on values, and the synergetic effect of Airbnb performance and neighborhood desirability. Although it does not have coefficients, it explains 0.709 test R-squared of neighborhood value variance with only a 6.5% train-test gap and is thus preferred for city planners.

The factors that affect the desirability of Chicago neighborhoods are based on the performance of Airbnb, historical home value trends, high-risk business density, and violent crime rates. This indicates that the desirability of a Chicago neighborhood can be improved by improving the performance of Airbnb, the historical trends of home value, the density of high-risk businesses, and the rates of violent crime, especially assault and robbery. This indicates that the desirability of a Chicago neighborhood can be improved by improving the performance of Airbnb, the historical trends of home value, the density of high-risk businesses, and the rates of violent crime, especially assault and robbery.

In the future, the model could be improved by adding more neighborhood-level factors, such as public transit access, school quality, walkability, and demographic information. Including these features would likely make the predictions more accurate and help city planners understand what drives home values in different types of neighborhoods. Testing the model in other cities could also show whether these patterns hold more broadly, and overlaying equity or displacement risk could help make planning decisions fairer and more informed.

In conclusion, the XGBoost model predicts Chicago home values with an R-squared of 0.709 on the test data and an average error of approximately \$65,574 per neighborhood. Airbnb performance, historical home values, commercial activity, and violent crime are the primary drivers of neighborhood desirability. These findings provide clear guidance for city planners: monitoring Airbnb trends, targeting violent crime reduction, and managing commercial development can directly influence neighborhood value and support strategic urban planning decisions across Chicago.

Act as a senior data scientist and provide critical feedback on this project. Review my 1-page write-up for clarity, structure, and impact. Review my Python code for clarity, efficiency, and best practices. Suggest specific improvements for both.”

UNEDITED FEEDBACK: “Here’s focused, “senior DS” feedback split into (1) write-up and (2) code, with concrete edits you can apply.

1. Write-up: clarity, structure, impact

Overall structure

Strengths:

- Clear project goal and data sources.
- You report concrete metrics and connect them to planning use cases.
- You already contrast RF vs XGBoost, which is exactly what a stakeholder cares about.

Key improvements:

- Tighten repetition and wordiness so it truly fits on one compelling page.
- Separate “Methods”, “Results”, and “Implications” more cleanly.
- Be more precise about what “desirability” means and what the model can and cannot imply (no causal claims).

Suggested revised outline (high level)

1. Goal & framing (2–3 sentences)

- Define neighborhood desirability and why you proxy it with ZHVI.
- State: “We use 17 features to predict average 2025 ZHVI across 53 community areas.”

2. Data & methods (1 short paragraph or bullet list)

- Data sources with years.
- Feature families (crime, food, parks, 311, Airbnb, price history).
- Mention train/test split and models (RF, XGBoost), plus how performance is evaluated (R^2 , MAE, RMSE).

3. Results (1–2 paragraphs)

- First: model performance comparison with numbers.

- Second: feature importance and qualitative interpretation.

4. Implications & limitations (1 paragraph)

- What city planners can use this for, and what they should *not* infer (no causality).
- Note sample size and omitted variables.

5. Future work (2–3 sentences)

- Additional features and external validation (other cities).

Concrete wording edits

Below are specific, surgical changes you can make to your text.

1. Opening paragraph:

Current:

“Neighborhood desirability is determined by the Zillow Home Value Index because home values are indicative of how safe, desirable, and economically viable an area is to its residents and investors.”

Suggested:

“Neighborhood desirability is proxied by the Zillow Home Value Index (ZHVI), which reflects how markets price safety, amenities, and economic opportunity for residents and investors.”

Also change:

- “53 community areas” (good) – but earlier you say “53 neighborhoods in Chicago”; make this consistent: call them **“53 Chicago community areas”** throughout.
- Remove several “neighborhoods” vs “community areas” flips.

2. Data paragraph:

Current:

“There were 53 neighborhoods in Chicago, with 39 used for training (75%) and 14 for testing (25%).”

Suggested:

“The final dataset included 53 community areas; 39 (74%) were used for training and 14 (26%) for testing.”

Also:

- You list violent crimes in features but never explicitly say which categories you included (you did in code). One concise sentence helps:

“Violent crimes were defined as homicide, robbery, assault, and battery.”

3. Model performance paragraph:

Current:

“It is clear that XG Boost performed significantly better than Random Forest. The XG Boost algorithm had a high R-squared value of 0.774 for training data and 0.709 for test data.”

Edits:

- Use consistent model name: “**XGBoost**”, not “XG Boost”.
- Be crisp about the interpretation of R^2 .

Suggested version:

“XGBoost substantially outperformed Random Forest. XGBoost achieved $R^2 = 0.77$ on the training data and $R^2 = 0.71$ on the test data, with a mean absolute error (MAE) of \$65,574 and RMSE of \$83,250, and only a 6.5 percentage point train–test gap.”

For Random Forest:

“Random Forest achieved $R^2 = 0.49$ on training and $R^2 = 0.32$ on test data, with test MAE of \$109,519 and a train–test gap of about 17 percentage points, indicating weaker generalization.”

Here are focused comments on the **code** side: readability, efficiency, and modeling practice.

1. Data prep & feature engineering

What’s strong

- Clear, stepwise construction of datasets (311/crime → parks/food → Zillow → Airbnb).

- Sensible engineered features: violent_crime_rate, food_safety_ratio, park_acres_per_1000_residents, home_value_growth_rate, etc. [[ppl-ai-file-upload.s3.amazonaws](#)]
- You're careful with types and NaNs (e.g., astype(str).str.zfill(5), errors='coerce'). [[ppl-ai-file-upload.s3.amazonaws](#)]

Improvements

1. Modularize with functions

Right now everything is in one script-like flow. For reuse and clarity, wrap major blocks in functions:

python

```
def load_chicago_core():
```

```
    df_311 = pd.read_csv("311_Service_Requests_20260128.csv", low_memory=False)
    df_crime = pd.read_csv("Crimes_-_2001_to_Present_20260128.csv", low_memory=False)
    df_food = pd.read_csv("Food_Inspections_20260128.csv", low_memory=False)
    df_parks = pd.read_csv("CPD_Parks_20260128.csv", low_memory=False)

    return df_311, df_crime, df_food, df_parks
```

Similarly: build_crime_311_features, build_food_parks_features, build_airbnb_zillow_features, build_final_dataset.

2. Avoid duplicative operations

Example: crime_311_ca = crime_by_ca.merge(...) is assigned twice before printing. Drop the second assignment. [[ppl-ai-file-upload.s3.amazonaws](#)]

3. Clarify “per 1000 residents” features

park_acres_per_1000_residents is actually per 1000 restaurants, not per 1000 residents:

python

```
final_clean_df['park_acres_per_1000_residents'] = (
    final_clean_df['park_acres'] / final_clean_df['restaurant_count'] * 1000
)
```

Either:

- Rename to something like park_acres_per_1000_restaurants, or
- Replace the denominator with an actual population field if you can get it.

4. Be explicit about violent crime definition

The lambda in violent_crimes is great but opaque to readers. Add a comment or refactor:[[ppl-ai-file-upload.s3.amazonaws](#)]

python

```
VIOLENT_TYPES = ['HOMICIDE', 'ROBBERY', 'ASSAULT', 'BATTERY']
```

```
crime_by_ca = df_crime.groupby('Community Area').agg(
    crime_count=('ID', 'count'),
    violent_crimes=('Primary Type', lambda x: x.isin(VIOLENT_TYPES).sum())
).reset_index()
```

5. Address multicollinearity explicitly

You correctly flag pairs with $|r| > 0.7$. [[ppl-ai-file-upload.s3.amazonaws](#)] For a tree model it's not fatal, but you could:

- Drop one of each highly correlated pair from X to simplify feature importances, e.g.:
 - total_crimes vs violent_crimes → keep one (probably violent).
 - restaurant_count vs high_risk_businesses vs service_request_density vs airbnb_listings_count.
- Or at least comment which ones you considered redundant and why you chose the final 8 features.

2. Modeling code (RF & XGBoost)

What's strong

- You do a **train/test split** and report metrics separately. [[ppl-ai-file-upload.s3.amazonaws](#)]
- You run simple hyperparameter sweeps and plot R^2 /MAE vs depth, vs n_estimators. [[ppl-ai-file-upload.s3.amazonaws](#)]

- You compute **5-fold CV** for both RF and XGBoost (good sanity check on stability).[\[ppl-ai-file-upload.s3.amazonaws\]](#)
- Feature-importance plotting is clear and labeled.[\[ppl-ai-file-upload.s3.amazonaws\]](#)

Improvements to structure & style

1. Use train_test_split outputs directly (avoid extra np.array casts)

You convert to numpy manually:

```
python
```

```
X_train = np.array(X_train)
```

```
y_train = np.array(y_train)
```

```
...
```

This isn't necessary; scikit-learn handles pandas DataFrames fine and preserves column names for inspection. You can drop those lines and keep X_train, X_test as DataFrames.

2. Factor out repeated metric calculation

You compute R²/MAE/RMSE multiple times with repeated code. Extract a helper:[\[ppl-ai-file-upload.s3.amazonaws\]](#)

```
python
```

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

```
import numpy as np
```

```
def regression_metrics(y_true, y_pred, prefix=""):
```

```
    r2 = r2_score(y_true, y_pred)
```

```
    mae = mean_absolute_error(y_true, y_pred)
```

```
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
```

```
    print(f"{prefix}R2={{r2:.4f}}, MAE={{mae:,.0f}}, RMSE={{rmse:,.0f}}")
```

```
    return r2, mae, rmse"
```


Reflection on feedback

The feedback was actually really helpful in showing me how to make my write-up clearer and more useful for someone who might actually use it. I went ahead and incorporated a lot of the suggestions, such as breaking up the Methods, Results, and Implications sections so it's easier to follow, removing repetitive wording, and making sure I'm really clear about what neighborhood desirability actually means and what the model can and can't tell us. I also changed how I discuss the metrics, such as R-squared, MAE, and RMSE, so it's more about what they mean in the real world and less about technical details such as max depth or estimators. For the features, I concentrated on the ones that actually matter instead of listing all of them, so it's easier for someone like a city planner to understand.

A few things I chose not to do are mostly related to the coding side of things. I didn't break everything down into small functions or try to address each individual problem with multicollinearity because the focus wasn't to write production-quality code but to keep things simple and easy to understand. I also didn't try to add more features such as population-normalized parks or additional crime categories because it would just make things more complicated without really changing what I learn from the model. The feedback has helped me determine what's important, how to explain it so it makes sense, and how to keep things simple and practical.